SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	S Susmitha	**Reg. No.:**	192571068
Section / Batch:	Slot B	**Date:**	27.07.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Code of Conduct

I S Susmitha (192571068) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____  _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SIMATS ENGINEERING

CO2 - ASSESSMENT TOOL 3

Name	Register Number	Course Code	Course
S Susmitha	192571068	CSA0603	Design and Analysis of Algorithms

QUESTION

Debugging Maximum–Minimum Finder Code (Improper Initialization for Negative Inputs)

The following code attempts to find both minimum and maximum values in an array but fails for all-negative datasets due to incorrect initialization. Carefully identify the root cause of the problem and explain why the output becomes invalid. Debug the implementation step-by-step to correctly handle positive, negative, and mixed inputs. Optimize the logic for clarity and correctness without changing the brute force nature of the solution. Analyze the time complexity of the corrected algorithm. Rewrite the final code with proper documentation.

def min_max(arr):

mn = 0

mx = 0

for x in arr:

if x < mn:

mn = x

if x > mx:

mx = x

return mn, mx

EXECUTING THE GIVEN CODE

Given Code (with Logical Error):

```
def min_max(arr):
    mn = 0
    mx = 0
    for x in arr:
        if x < mn:
            mn = x
        if x > mx:
            mx = x
    return mn, mx

arr = [-8, -3, -10, -5]
print(min_max(arr))
```

Incorrect Output:

```
>>>
=== RESTART: C:/Users/Susmitha/Downloads/CSA0603 Assessment/CO2 AT3 Error.py ===
(-10, 0)
```

Explanation:

Error Identified:

The program executes successfully, but produces an incorrect output for arrays containing only negative numbers. This is a **logical error** caused by initializing both the minimum (mn) and maximum (mx) variables to 0. Since no negative number is greater than 0, the maximum value (mx) is never updated, causing the program to incorrectly return 0 as the maximum value even though 0 is not present in the array.

For the input:

arr = [-8, -3, -10, -5]

the program outputs:

(-10, 0)

Instead of the correct output:

(-10, -3)

Time Complexity

- Time Complexity: **O(n)**
- Reason: The algorithm **traverses the array once** using a single for loop.

FIXING THE CODE

STEP 1: CORRECT THE MINIMUM INITIALIZATION

Code:

```
def min_max(arr):
    mn = arr[0]
    mx = 0
    for x in arr:
        if x < mn:
            mn = x
        if x > mx:
            mx = x
    return mn, mx


arr = [-8, -3, -10, -5]
print(min_max(arr))
```

Output:

```
>>>
======================================== RESTART: C:/Users/Susmitha/Downloads/CSA0603 Assessment/CO2 AT3 Fix 1.py
(-10, 0)
```

Explanation:

Error Identified

Although the minimum value is correct for the given input, initializing mn to 0 is not a good programming practice because it assumes that 0 is an appropriate starting value. Initializing mn with the first element of the array ensures the algorithm works correctly for all datasets.

Fix Applied

The minimum variable was initialized using the first element of the array instead of 0.

Changed from

mn = 0

to

mn = arr[0]

Output

(-10, 0)

The output remains unchanged because the main issue still lies with the maximum variable (mx), which is still initialized to 0.

Time Complexity

- Time Complexity: **O(n)**
- Reason: Only the initialization value was changed. The algorithm **still scans the array once.**

STEP 2: CORRECT THE MAXIMUM INITIALIZATION

Code:

```
def min_max(arr):
    mn = arr[0]
    mx = arr[0]
    for x in arr:
        if x < mn:
            mn = x
        if x > mx:
            mx = x
    return mn, mx

arr = [-8, -3, -10, -5]
print(min_max(arr))
```

Output:

```
>>>
========================================= RESTART: C:/Users/Susmitha/Downloads/CSA0603 Assessment/CO2 AT3 Fix 2.py
(-10, -3)
```

Explanation:

Error Identified

The maximum variable is still initialized to 0.

mx = 0

For an array containing only negative numbers, none of the elements are greater than 0, so the maximum value never changes from its initial value.

Fix Applied

The maximum variable was initialized with the first element of the array.

Changed from

mx = 0

to

mx = arr[0]

Output

(-10, -3)

The program now correctly finds both the minimum and maximum values for negative, positive, and mixed datasets.

Time Complexity

- Time Complexity: **O(n)**
- Reason: The algorithm still performs a **single traversal of the array**. Changing the initialization does not affect the number of iterations.

STEP 3: HANDLE EMPTY ARRAYS

Code:

```
def min_max(arr):
    if len(arr) == 0:
        return None

    mn = arr[0]
    mx = arr[0]

    for x in arr:
        if x < mn:
            mn = x
        if x > mx:
            mx = x

    return mn, mx

arr = [-8, -3, -10, -5]
print(min_max(arr))
```

Output:

```
>>>
========================================= RESTART: C:/Users/Susmitha/Downloads/CSA0603 Assessment/CO2 AT3 Fix 3.py
(-10, -3)
```

Explanation:

Issue Identified

If an empty array is passed to the function, accessing arr[0] will cause an **Index Error** because there is no first element.

Fix Applied

An empty-array check was added before accessing the first element.

```
if len(arr) == 0:
    return None
```

This makes the program more robust and prevents runtime errors when the input list is empty.

Output

(-10, -3)

The output remains correct, and the function is now capable of safely handling empty input as well.

Time Complexity

- Time Complexity: **O(n)**
- Reason: The added empty-array check takes **O(1)** time. The algorithm still **traverses the array only once**, so the overall complexity remains **O(n)**.

FINAL CORRECTED CODE:

```
def min_max(arr):
    if len(arr) == 0:
        return None
    mn = arr[0]
    mx = arr[0]
    for x in arr:
        if x < mn:
            mn = x
        if x > mx:
            mx = x
    return mn, mx
arr = [-8, -3, -10, -5]
print(min_max(arr))
```

ALGORITHM, OPTIMIZATION AND EFFICIENCY

ALGORITHM (Brute Force - Linear Scan)

Input: arr = [-8, -3, -10, -5]

Output: Minimum = -10, Maximum = -3

Algorithm

1. Start the algorithm.
2. Read the input array arr = [-8, -3, -10, -5].
3. Check whether the array is empty.

   If the array is empty, return None and stop.

4. Initialize:

mn = arr[0] = -8

mx = arr[0] = -8

5. Traverse each element of the array:

Element = -8

-8 < -8 → False

-8 > -8 → False

mn = -8, mx = -8

Element = -3

-3 < -8 → False

-3 > -8 → True → mx = -3

mn = -8, mx = -3

Element = -10

-10 < -8 → True → mn = -10

-10 > -3 → False

mn = -10, mx = -3

Element = -5

-5 < -10 → False

-5 > -3 → False

mn = -10, mx = -3

6. Return the values:

Minimum = -10

Maximum = -3

7. Stop the algorithm.

OPTIMIZATION PERFORMED

The brute-force algorithm was **not changed**. Instead, the implementation was optimized by improving its correctness and reliability:

- Initialized mn with arr[0] instead of 0.
- Initialized mx with arr[0] instead of 0.
- Added an empty-array check (if len(arr) == 0:) to prevent an IndexError.
- Improved code clarity while preserving the original brute-force approach.

These changes optimize the **logic and robustness** of the program without changing its algorithmic complexity.

EFFICIENCY

Technique Used: Brute Force (Linear Traversal)

Time Complexity: O(n)
The array is **traversed only once**, and each element is compared with the current minimum and maximum values.

Space Complexity: O(1)
Only two additional variables (mn and mx) are used, so the **extra memory required is constant.**

CONCLUSION

The original program contained a **logical error** because **both the minimum and maximum variables were initialized to 0**, causing incorrect results for arrays containing only negative numbers. By **initializing both variables with the first element of the array and adding an empty-array check**, the algorithm now correctly handles positive, negative, mixed, and empty arrays while preserving the brute-force approach with a time complexity of **O(n)**.